

Photonic Quantum Computing on Spin Memory Architecture with Tree-Encoded Fusion

Xiangyu Ren[†], Yuexun Huang[‡], Zhemin Zhang[§], Yuchen Zhu^{||},
Tsung-Yi Ho[§], Antonio Barbalace[†], Zhiding Liang^{§*}

[†] University of Edinburgh, Edinburgh, Scotland, UK

[‡] University of Chicago, Chicago, IL, USA

[§] The Chinese University of Hong Kong, Sha Tin, Hong Kong

^{||} Northwestern University, Chicago, IL, USA

Abstract—Photonic quantum computer (PQC) is a promising quantum computation platform, realizing the measurement-based quantum computation (MBQC) model. In MBQC, computation proceeds by preparing a graph state, and this preparation mainly relies on fusion operations. However, fusion operations on a PQC are prone to two types of errors: fusion *failure* and fusion *erasure*. Therefore, the MBQC compiler must be carefully designed to tolerate these errors. Previous state-of-the-art MBQC compiler – OneAdapt, is tailored to all-photonic architectures and primarily address fusion failures. However, it neglects fusion erasure errors caused by photon loss, which are more detrimental than fusion failures.

To address the challenge of fusion erasure, we propose a novel MBQC scheme that is based on *quantum spin memory* architecture. We design a tree-encoded fusion scheme that effectively suppresses erasure errors. Then, we integrate this scheme into our compiler framework, with compilation algorithms that reduce execution overhead of quantum programs. We evaluate our framework on a realistic PQC simulator across six typical quantum algorithm benchmarks with various program sizes. Our results show that the proposed tree-encoding scheme outperforms other fusion encoding schemes, and that our compilation framework outperforms OneAdapt with exponential improvement. Moreover, we demonstrate a small-scale QAOA experiment on real PQC hardware, where it outperforms the latest superconducting hardware.

Index Terms—photonic quantum computing, quantum compilation, fault-tolerant quantum computation.

I. INTRODUCTION

Photonic quantum computer (PQC) is one of the highly potential quantum computing systems that pave the way to quantum supremacy [48]. Operated at room temperature, the photonic qubits have long decoherence time and great scalability [56]; also, their natural distributed characteristic makes PQC easy for quantum network integration [9], [71]. Various companies have demonstrated their PQC hardware platforms: PsiQuantum introduces their manufacturable platform of PQC with 99.72% two qubit fusion fidelity [52], and Quandela provides cloud platform access with a 24-photon modes PQC system [2]. Despite the rapid progress of photonic quantum hardware, compilation techniques of photonic quantum computer (PQC) is yet to be fully explored, while there are only a few novel PQC compilers [72], [74] evaluated on a simplified quantum simulation model.

MBQC and graph state. Different from other typical quantum computing hardware (e.g., superconducting or neutral-atoms), photonic quantum computing adapts a measurement-based computation (MBQC) model [10] rather than the usual gate-based model. In the paradigm of measurement-based quantum computation, a quantum program is represented as a highly entangled quantum state – the graph state, and sequential measurements are performed on qubits of the graph state to execute corresponding computations. Hence, the graph state plays a key role in MBQC, and the main challenge of MBQC compilation on photonic systems is to generate the target graph state robustly and efficiently.

Fusion operations for graph states are imperfect. In a photonic system, photonic qubits are emitted from the photon sources, and they are entangled through *fusion* operations to form a graph state. Fusion operations are prone to two types of errors: *fusion failure* and *fusion erasure*. Fusion failures arise from the natural characteristic of photonic qubits. When the fusion of two qubits fails, they are automatically measured out and have no impact on the rest of their entangled qubits. In contrast, fusion erasure is caused by photon loss: one of the fusion qubits cannot be detected by the system, thus the outcome of fusion operation remains unknown. In real PQC hardware, we observe an erasure rate $p_{eras} \approx 10\%$. This level of erasure error is detrimental to the execution of quantum programs. Details are explained in Sec. III-A.

Prior Works. A recent state-of-the-art MBQC compiler – OneAdapt [74] purpose an efficient framework to execute quantum programs on PQC, and it tackles the fusion failure by utilizing a normalization method. However, it neglects the occurrence of fusion erasure, which is unable to deal with only using normalization. Also, normalization method distillates a valid 2D graph state layer from much larger number of photonic qubits, which fails to efficiently utilize photon resources. Meanwhile, Li et al. [38] propose the RLGS compiler on emitter-based photonic architecture. It benefits from the deterministic nature of emitter-based photon source, and utilize the interactions between emitters to generate entanglements for graph state. Despite this, emitter-based solution has still not been fully demonstrated in experiment, due to the challenge of coupling between emitters efficiently [25]. As a result, the evaluations in [38] are based on assumptions from theoretical

analysis [19], [57], so the proposed solution still has a gap from near-term photonic quantum computing systems.

Quantum spin memory architecture. To fill this gap, we introduce our PQC framework to handle the fusion errors in the near-term PQC architecture with a realistic hardware model. The PQC architecture that we adapt [25] utilizes silicon-based *quantum spin memory* as photon source, which could generate a specific structure of resource graph state, namely *caterpillar* state. The caterpillar state has been demonstrated experimentally in a small scale [29]. Next, these caterpillar states are concatenated together by fusion operations, forming a large graph state corresponding to the target quantum program. Generally, the main problem we are addressing in this work, is finding an efficient solution that generates the target graph state from caterpillar state, while maintaining robustness against fusion failure and erasure.

Firstly, we propose a **novel tree-encoded scheme to boost the success rate of fusion operation**, while effectively suppressing fusion failure and fusion erasure. We leverage the flexibility of caterpillar state, and are inspired by the quantum error correction (QEC) tree code [1], [7], [68] to design our unique fusion scheme. Specifically, we eliminate the photonic qubit that undergoes erasure (photon loss) utilizing the *indirect Z-measurement*. Meanwhile, our dedicated tree structure design provides multiple trials for the fusion operation, offering resistance to fusion failures. Compared to previous boosted-fusion schemes, e.g. repetition encoding [25] and repeat-until-success [21] approach, our tree-encoded scheme achieves significant error-tolerance against high erasure error rates (Fig. 4(f)). Furthermore, the tree-encoding structure can be easily generated from the caterpillar state, proving its fitness for quantum spin memory architecture. Detailed theoretical analysis and simulation are given in Sec. IV.

Secondly, we propose **MemTree** – a **compilation framework for scalable and resource-efficient execution** of quantum programs on quantum spin memory architecture. We implement a divide-and-conquer algorithm to separate the target graph state into multiple caterpillar states while reducing the total number of fusions to minimize execution overhead. Specifically, we design a hierarchical minimal-cut algorithm for dividing the target graph state. In the algorithm, we leverage mix-integer-programming (MIP) to obtain the solution, with dedicated constraints to guarantee the divided subgraphs conform to the structure of caterpillar states. Details of compiler design are given in Sec. V.

Thirdly, we build a **realistic error-aware simulator** to evaluate our framework and compare it with SOTA [74]. Our simulator considers the impact of fusion failure and fusion erasure based on the hardware noise model, setting the configurations from experimental works [6], [29], [43], [52], [66]. We choose two dominant overheads of photonic MBQC as metrics: (1) Total execution time of the quantum program. (2) Number of photon sources required to perform the computation. In addition, our simulation considers the errors in preparing tree-encoded logical qubits, ensuring a realistic evaluation of the fault-tolerance scheme. Details are

given in Sec. VI.

Lastly, we perform the **evaluation on a comprehensive set of benchmarks**, which includes 6 quantum algorithms, each with varying sizes (36-100 qubits). We evaluate our tree-encoded fusion scheme against previous boosted fusion scheme – repetition-encoded fusion [25] and repeat-until-success fusion [40], where our scheme reduces the program execution time by a factor of 1.9×10^{-3} and 1.7×10^{-2} , respectively. We compare our PQC framework with OneAdapt, and on average, we achieve reduction rates of 1.5×10^{-2} in execution time, $0.18\times$ in photon resources, $0.14\times$ in compilation runtime, and a $3.64\times$ improvement in fidelity. Compared with RLGS [38], we achieve an improvement of $1.42\times$ in fidelity. In addition, we demonstrate the QAOA algorithm on real hardware of Quandela PQC platform [2].

Our contributions are listed as follows:

1. We consider a more realistic error mechanism in photonic quantum computing and propose a tree-encoded fusion scheme to protect against fusion failure and erasure, while the latter error has been overlooked by previous works [72], [74], [75].
2. We tailor the tree-encoded fusion scheme to a novel PQC architecture – the spin memory architecture, and design a compilation framework that reduces program execution time while minimizing photon resource overhead.
3. We implement a realistic simulator of spin memory architecture, which is based on configurations of a successfully demonstrated hardware platform [29]. We compare our work with other fusion schemes [21], [26] and SOTA PQC compilers [74], while the results show significant improvements in execution time and resource overhead.
4. To the best of our knowledge, prior compiler works for photonic MBQC have been evaluated primarily in simulation, whereas our work includes real-hardware validation through a quantum algorithm demonstration, and shows improvement compared to prevalent superconducting hardware.

II. BACKGROUND

A. MBQC Background

The Graph state is a special type of multipartite entangled state [23], with its intrinsic structure determined by a graph $G = (V, E)$, where V denotes the set of vertices and E denotes the set of edges. Then, each vertex $v \in V$ is associated with a qubit. The formal definition of a graph state $|G\rangle$ is given by

$$|G\rangle = \prod_{(i,j) \in E} CZ_{(i,j)} |+\rangle^{\otimes V}, \quad (1)$$

where $|+\rangle^{\otimes V}$ is the tensor product state with all $|V|$ qubits initialized in the X eigen-state $|+\rangle$, and $CZ_{(i,j)}$ represents the CZ gate applied to qubits(vertices) i and j connected by edge (i, j) in G . The graph state is shown to be universal for MBQC [53], in the sense that only single qubit measurements are required for any computation once the graph state is generated. We refer to the background sections of [72], [74] for more details about graph states and MBQC.

Fusion is arguably the most important operation in graph state generation, as it allows us to combine smaller graph

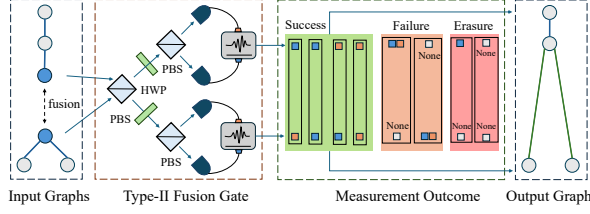


Fig. 1. Type-II fusion operation. Two qubits (each from one input graph) are performed fusion operation, combined into a larger output graph. The fusion is success or not depends on the measurement outcome of these two qubits: if they are captured in different sides of detectors, the fusion succeeds; if captured at the same side, the fusion fails; if one of the qubits not captured, it leads to fusion erasure.

states to form a desired larger target graph state, thus enabling resource efficient parallel generation [11], [28]. In the *Type-I fusion*, the two vertices entering the fusion gate from the two smaller graph states are merged into one, inheriting the edges from both and resulting in a larger graph state. Whereas the result for *Type-II fusion* is a bit more complicated: both input vertices are removed from the graph, while the neighbors of one vertex are connected to (disconnected from) the neighbors of the other vertex if they were previously disconnected (connected). Both types of fusion can be implemented probabilistically via linear optics using half-wave plates (HWP) and polarizing beam splitters (PBS). And we focus on Type-II fusion in this work, as its photon loss can be heralded [37], for which an intuitive example¹ is provided in Fig. 1.

B. PQC Hardware Architectures

The key and most difficult part of generating the graph state is constructing the required CZ connections (edges) between the qubits (vertices). There are mainly three fundamental hardware architectures for generating the resource graph states for PQC, as illustrated in Fig. 2(a)-(c).

In the **all-photonic** architecture, the required equipment consists of linear optical elements and the conventional spontaneous parametric down-conversion (SPDC) source for photonic bell pair generation [11], [34], [78]. The generated photonic bell pairs can be merged together to form a larger graph using the aforementioned fusion operation. However, such a fusion process is probabilistic, with a 50% chance at best (which can be enhanced to 75% by additional optical hardware [18], [22]). As illustrated in Fig.2(a), a graph state with any underlying topology can be generated by repeating the above fusion process with sufficient resource bell pairs [35], which is the scheme explored in OneAdapt [74].

In contrast, the **emitter-based** architecture theoretically promises a deterministic generation of graph states through the use of interacting quantum emitters [17], [20], [41], [58]. The proposal arises from two basic mechanisms shown in Fig. 2(b): (1) The emitted photon from a quantum emitter is

¹The implementation of type fusion varies across different literature depending on whether HWP are inserted before the first PBS [11], [26], [35], resulting in different local unitary corrections required upon success and different effective measurements upon failure. We adapt the scheme in [25].

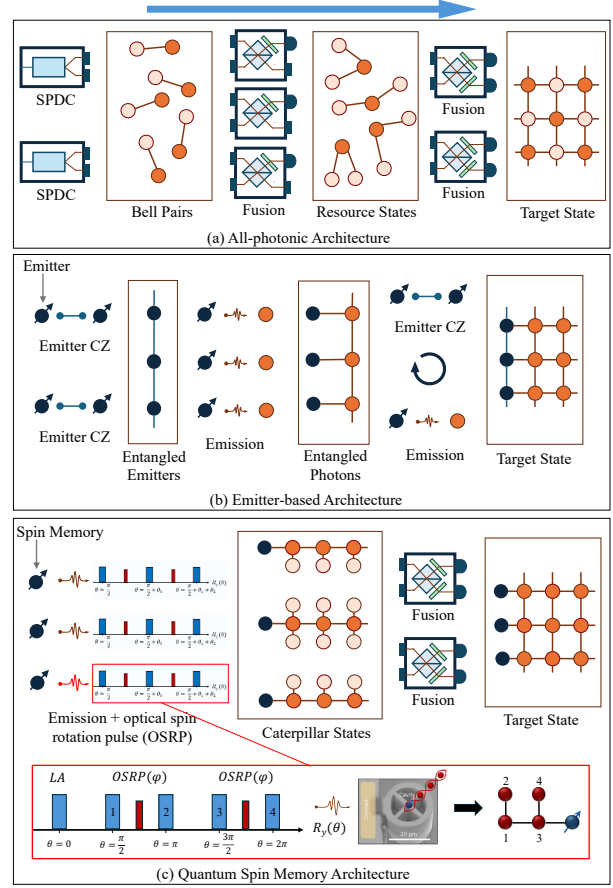


Fig. 2. The comparison among different PQC architecture and their corresponding graph state generation schemes. The excitation pulses for generating a caterpillar state are demonstrated in the red box of (c). Specifically, longitudinal-acoustic excitation $LA(\frac{\pi}{2})$ and optical spin rotation pulses ($OSRP(\varphi), \varphi = \pi$) are applied to QD-cavity [30] in dedicated sequence, emitting the target caterpillar graph shown on the right.

entangled with this emitter, resulting in an effective CNOT gate between the emitter qubit and the photon qubit. (2) The emitters themselves can be entangled with each other by implementing a CZ gate, which is theoretically analyzed in [57], but has not yet been experimentally demonstrated. These mechanisms result in a series of basic generation rules, based on which an arbitrary graph state can be generated [33], [36], [54]. RLGS [38] is the SOTA compiler framework based on this architecture.

The **quantum spin memory** is a PQC architecture [12], [21], [30], [51] based on semiconductor quantum dot (QD) emitters. The preparation of a caterpillar state is illustrated in the red box of Fig. 2(c). When applying longitudinal-acoustic (LA) excitation pulses on the QD-cavity iteratively, it can emit linearly entangled photons as graph states. Additionally, we can interleave optical spin rotation pulses (OSRP) into the excitation pulses, leading to the emission of a special graph state structure – the *Caterpillar* state [51]. As shown in Fig. 2(c), *Caterpillar* state has a branched-chain structure,

with a chain of linearly entangled qubits as the *main path*, and extra *leaf qubits* each directly connected to one qubit of the *main path*. We refer to [51] for its complete definition and [30] for its physical preparation process. Next, the caterpillar states are concatenated into the target graph states through a set of fusion operations using linear optical hardware similar to an all-photonic architecture. While spin memory architecture is also prone to fusion errors, the flexibility within the caterpillar structure enables us to design and integrate error-tolerant encoded graph states (details in Sec. IV).

III. MOTIVATION

A. Errors in Fusion Operation

In type-II fusion, the two dominant error sources are *fusion failure* and *fusion erasure*. Although both arise from the same imperfect fusion primitive, they differ in a key way: fusion failure leads to a *known* graph transformation, whereas fusion erasure leads to an *unknown* graph outcome.

As shown in Fig. 1, **fusion failure** is heralded when two fusion qubits are captured in the same detector, indicating that the desired entanglement is not created. In this case, the failed qubits are effectively measured in the Z basis and disconnected from the graph. Therefore, although the fusion attempt is unsuccessful, the resulting graph structure remains known to the compiler. This is the failure model considered in previous compilers such as OneAdapt [74] and OnePerc [72].

In contrast, **fusion erasure** is triggered by photon loss during fusion, where one fusion qubit cannot be captured by the detector, as shown in Fig. 1. The erased qubit is no longer accessible for computation, and its effect cannot be removed by a direct Z measurement. More importantly, the output graph state of the fusion becomes uncertain, since it is unknown whether the entanglement has been established or not. Such uncertainty is especially harmful to MBQC, because later measurements rely on the exact graph-state structure; therefore, the corrupted fusion output must be discarded unless additional protection is applied.

These two errors are also closely related in boosted-fusion design. A common way to suppress fusion failure is to introduce more fusion attempts, but each extra attempt also exposes more qubits to photon loss and thus increases the chance of fusion erasure. As a result, improving tolerance to fusion failure alone is insufficient under realistic photon-loss conditions. To demonstrate the impact of erasure, Fig. 3 shows a simulation of a Max-Cut QAOA program under different erasure rates using the previous best fusion scheme. The results show that erasure undermines quantum programs in two aspects: (i) it increases the number of fusion attempts, leading to exponentially longer execution time; (ii) the longer execution time accumulates higher decoherence and CZ errors, resulting in lower-quality outputs and larger program-level overhead, such as more tuning iterations in QAOA.

B. Problems in Previous SOTA Compilers

OneAdapt [74] and RLGS [38] are the SOTA compilers for all-photonic and emitter-based architectures, respectively.

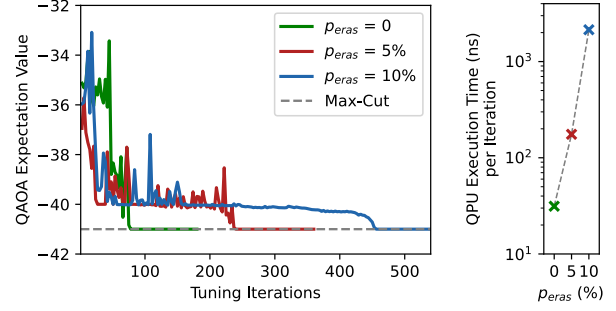


Fig. 3. Optimizing a Max-Cut problem using 6-qubit QAOA program on PQC simulator [24]. We use the RUS boosted fusion method ($m = 6$), and simulate the fusion erasure at 0, 5% and 10% respectively, while fixing the fusion failure at 25%. Left: Optimization of QAOA expectation value. Right: Quantum circuit execution time per tuning iteration.

Though carefully designed, there are still several gaps from implementing the realistic error-tolerant MBQC, and we conclude their existing problems in Table. I.

OneAdapt iteratively generates resource state layers (RSL) and normalizes them into effective 2D layers of lattice graph states to create the target graph state of a quantum program. This strategy resolves the fusion failure problem, but it overlooks the fusion erasure, which induces errors in the 2D graph state layers. Assuming an 1% erasure rate, generating the required 84×84 RSL will demand $> 10^5$ fusion operations, leading to an extremely low probability of not experiencing erasure in the whole RSL. Furthermore, the normalization method results in a low utilization rate of photons. For example, OneAdapt normalizes only a 4×4 2D layer from the 84×84 qubits RSL [74].

The problem of **RLGS** primarily lies in the bottleneck of emitter-based architecture hardware. Up to now, only the generation of linear graph states with few qubits from a single quantum emitter has been experimentally demonstrated [13], [59]. The hardware bottleneck arises from the inability to demonstrate high-quality CZ interactions between two emitters [26], which is essential for generating MBQC graph states.

C. Potentials in Spin Memory Architecture

We list our insights on addressing the above challenges in the MBQC compiler, leveraging the spin memory architecture:

(1) The caterpillar state structure offers the chance to resist fusion failure by embedding specific graph state patterns, known as the boosted fusion scheme [25]. Since the above scheme failed to deal with fusion erasure, in this work, we explore the graph state pattern that tolerates both failure and erasure, while tailoring the pattern to the caterpillar state (Sec. IV).

(2) Equipped with the error-tolerant graph state pattern, we can improve the fusion success rate. As a result, we have no need for excessive photon sources and apply normalization like OneAdapt. In contrast, we arrange the generation of caterpillar states from photon sources to be *program-agnostic*, while the structure of each caterpillar state is on demand and determined

Architecture	Photonic Hardware	Error Types	Existing Problem	Prior Compiler Framework
all-photonic	linear optics	• fusion failure • fusion erasure	• Low utilization rate of photons • Fusion erasure error unsolved	• OneAdapt [74], • FCM [46]
emitter-based	quantum emitter	• emitter decoherence • emitter-CZ infidelity	Bottleneck of experimentally demonstrating the emitter-CZ	• RLGS [38], • GSDiv [55]
spin memory	quantum spin memory + linear optical hardware	• fusion failure • fusion erasure	Fusion erasure error unsolved	None

TABLE I
COMPARISON BETWEEN PHOTONIC QUANTUM COMPUTING ARCHITECTURES.

by the target graph state. Hence, we can improve the utilization rate of photon sources, with details in Sec. V.

(3) Our compiler considers the hardware settings from real experiments, gaining more robustness and being more achievable in near-term PQC. Compared to the emitter-based architecture, which still has unsolved hardware barriers, the spin memory architecture is accessible on cloud platforms [2], [24].

IV. A ROBUST TREE-ENCODING FUSION SCHEME

A. Previous Boosted-Fusion schemes

First, we briefly describe prior error-tolerant fusion schemes on spin memory architecture and analyze their problems.

Redundantly-encoded fusion. Hilaire et al. [25] introduce a fusion failure-tolerant protocol to improve the success rate. Utilizing the feature of the caterpillar state, the redundantly-encoded scheme generates a logical linear graph state. Each node of this linear graph is a logical qubit encoded by m leaf qubits. For the fusion of two logical qubits from different graph states, a fusion operation is applied to each pair of leaf qubits from these two logical qubits. As a result, m attempts are performed between the two logical qubits, while any single successful attempt leads to fusion success. The redundant fusion operations reduce the overall rate of fusion failure, however, they lead to more exposure to fusion erasure. Assuming p_{fail} and p_{eras} as the probabilities of fusion failure and erasure, respectively, an m -qubit redundantly-encoded fusion has the following logical error rates:

$$P_{fail} = p_{fail}^m, P_{eras} = 1 - (1 - p_{eras})^{2m}.$$

P_{eras} has a $2m$ exponential because any one of the m physical qubits in each logical qubit is exposed to erasure error independently [12], [21]. Here we can observe that when the redundant encoding parameter m grows, the logical failure rate P_{fail} reduces, but the logical erasure rate P_{eras} increases.

Repeat-until-success fusion. Lim et al. [40] propose a repeat-until-success (RUS) method to improve the fusion operation [21], [66]. In this RUS scheme, ancillary photons are used to apply fusion operations between two photon sources, thus creating entanglement between the caterpillar states produced by the two photon sources. The RUS scheme has a similar idea to the redundantly-encoded scheme, but it terminates once the fusion operations succeed. Its logical error rates are:

$$P_{fail} = p_{fail}^m, P_{eras} = \sum_{i=0}^{m-1} p_{fail}^i \cdot 2p_{eras}$$

Although the RUS scheme achieves slightly better performance than the redundantly-encoded scheme [25], it consumes more ancilla qubit resources and takes a longer time. Moreover, it has the same problem of intolerance to erasure errors.

B. Preliminary: Loss-Tolerant Graph State Patterns

Before introducing our tree-encoded fusion scheme, we explain the underlying principles. Our insight into tree-encoded fusion derives from the QEC tree graph-state code [1], [7], [68]. It reveals several properties of graph state measurement: Fig. 4(a) shows the direct Z measurement rule: A Z-basis measurement removes the target qubit from its graph state and breaks all the entanglement between the target qubit and all other qubits in the graph state. Fig. 4(b) shows the pair of X measurement rule: Two adjacent X-basis measurements on a linear cluster remove the qubits and form direct bonds between their neighbors. Most importantly, Fig. 4(c) shows the indirect Z measurement rule: we select a neighboring qubit i of the target qubit j_0 and perform an X measurement; then we perform Z measurements on all other qubits j_1, j_2 connected to this neighboring qubit i . These measurements deterministically reveal what the Z measurement outcome would have been on j_0 , based on the underlying stabilizer operator $X_i \prod_{j \in E(i)} Z_j$. Hence, if the target qubit j_0 that we want to measure undergoes photon loss, we can still measure it indirectly with this measurement pattern. We recommend [68] for complete details of the QEC tree code protocol.

C. Design of Tree-Encoded Fusion.

Inspired by both the redundantly-encoded fusion and the QEC tree code, we introduce our tree-encoded fusion scheme, illustrated in Fig. 4(d). Two logical qubits A and B involved in a fusion operation are encoded by a tree-structure: the root qubit q_{root} is connected to b branches, while each branch contains a linear graph of 3 qubits – $\{q_i^a, q_i^b, q_i^c\}$. The leaf qubit q_i^c is assigned for fusion measurement, while the other two qubits q_i^a, q_i^b are ancillary qubits for indirect measurement in case of fusion erasure. The specific operations are listed below for different outcomes of fusion:

- 1) Fusion success: perform the pair of X measurements on q_i^a and q_i^b , so that the successful fusion entanglement is directly connected to q_{root} .
- 2) Fusion failure: upon fusion failure, the q_i^c is measured out, while q_i^a, q_i^b remains in the tree. Then we apply the Z measurement on q_i^b to remove it and leave q_i^a for backup usage, as explained in (4).
- 3) Fusion erasure: if q_i^c undergoes erasure, we apply an X measurement on q_i^b and a Z measurement on q_i^a , leading

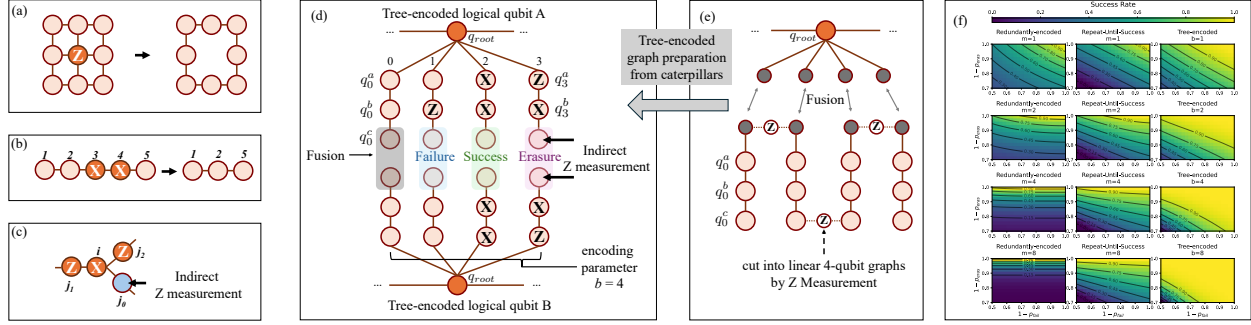


Fig. 4. (a)-(c) Graph state measurement patterns that establish loss-tolerance. (d) Tree-encoded fusion scheme. (e) Preparing tree-encoded logical qubit from caterpillar states. (f) Simulation of the fusion schemes. We compare these schemes with varying encoding parameters ($m, b = 1, 2, 4, 8$), by performing 10^3 fusion trials per data point to measure success rates.

to an indirect Z measurement on q_i^c . Such operations effectively eliminate q_i^c without impacting the rest of the qubits, hence forming erasure-tolerance.

- 4) In the extreme case that all branches of the tree end up with fusion failure or erasure, the backup q_i^a in (2) can still be used for a fusion attempt.

Owing to the protection of fusion erasure, the logical qubits A and B retain an entanglement when one of the branches i succeeds in fusion. It should be noted that our design of the tree-encoded scheme is a trade-off between the fusion success rate and photon resource consumption.

D. Analysis of Tree-Encoded Fusion.

Here, we analyze the fusion success rate of redundantly-encoded, RUS, and tree-encoded schemes under certain fusion failure and erasure probabilities (denoted by p_{fail} and p_{eras}). With m or b as the encoding parameter, we calculate their theoretical success rates as:

$$S_{redun} = (1 - p_{fail}^m) \cdot (1 - p_{eras})^{2m}$$

$$S_{rus} = 1 - \sum_{i=0}^{m-1} p_{fail}^i \cdot 2p_{eras} - p_{fail}^m$$

$$S_{tree} = 1 - (1 - (1 - p_{eras})^2 + p_{fail})^b$$

Here in S_{tree} , the probability of fusion erasure in each branch is calculated as $P_{eras} = 1 - (1 - p_{eras})^2$, since the probability of no erasure on each side is $1 - p_{eras}$.

In Fig. 4(f), we simulate the process of these fusion schemes, following the error probabilities and counting the average success rate. We show the success rate of the logical fusion operation, with $1 - p_{fail}$ and $1 - p_{eras}$ as the X and Y axes. It can be observed that our tree-encoded scheme outperforms the redundant-encoded scheme and RUS when the erasure rate p_{eras} scales up.

E. Tailoring Tree-Encoded Fusion to Spin Memory

The tree-encoded scheme can be generated from caterpillar states on a quantum spin memory architecture. In Fig. 4(e), we demonstrate the process of ensembling the tree-encoded logical qubit from caterpillar states. First, we generate the

caterpillar state that is used for the combination into the target state. Each logical qubit involved in later fusion is composed of q_{root} on the main path and b leaf qubits (in gray color) connected to q_{root} . Meanwhile, we generate b 4-qubit linear graph states, which can be separated from a long linear graph by using Z measurement. Then, we apply fusion to concatenate these linear graph states to the leaf qubits appended to q_{root} , and form the tree-encoded structure we need for fusion.

Since the preparation procedure for tree-encoded logical qubit is not protected by the scheme itself, preparations of these tree branches are exposed to fusion errors. To ensure a steady and robust preparation of a logical qubit with b -branching, we introduce a preparation parameter b_{prep} complying with $b_{prep} > b$. During the preparation, we perform b_{prep} attempts of branch preparation: (i) If fusion failure happens to a branch, it will be measured out automatically. (ii) If fusion erasure happens to a branch, we solve it by indirect measurement with Z-measurement on $\{q_i^b, q_i^c\}$. All the branch preparation are performed simultaneously in one timestep, if less than b branches are prepared successfully among b_{prep} attempts, it will be retried in the next timestep. In the next subsection we discuss an appropriate selection of b and b_{prep} under near-term PQC hardware limitation.

F. Tree-encoding Parameter

The selection of b and b_{prep} should ensure less preparation timestep and photon sources. In Fig. 5(b), we count the #photon sources required for varying b_{prep} . Given a 30-qubit limitation of caterpillar graph, #photon sources keeps steady and grows sharply when $b_{prep} > 6$. In Fig. 5(c), we evaluate the average execution time of MemTree on 36-qubit VQE and QFT programs. The results show that with an increasing value of b , execution time decreases on an exponential scale until $b = 4$, followed by convergence afterward.

Next, we study the relationship between b_{prep} and b in Fig. 5(a). Under the realistic fusion error rates of near-term PQC, when $b_{prep} = 6$ the preparation can ensure > 4 branches prepared averagely. Hence we select $b = 4$ and $b_{prep} = 6$ for a tradeoff between performance (fusion success rate) and

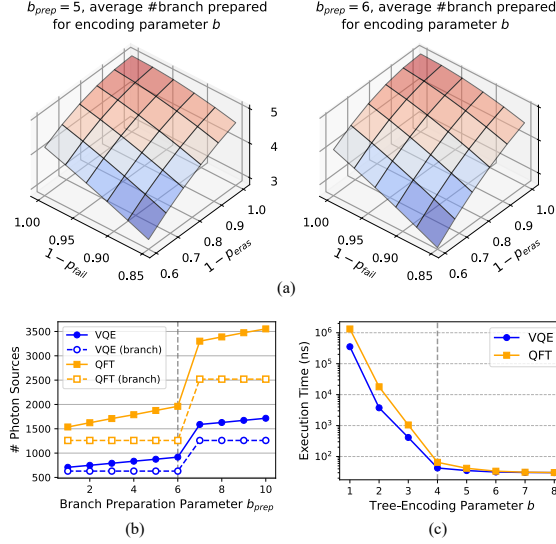


Fig. 5. (a) Average number of tree branches that successfully prepared for logical qubit encoding parameter b , when the preparation parameter $b_{prep} = 5$ and $b_{prep} = 6$ (by simulation). (b) Photon resource breakdown analysis for parameter b_{prep} , when given the maximum length of caterpillar is 30-qubit. Dashed lines represent the #photon sources used for branch preparation. (c) Execution time analysis for the tree-encoding parameter b , under a noise model that $p_{fail} = 2\%$ and $p_{eras} = 25\%$.

number of photon sources. Furthermore, in real hardware experiment (Sec. VII-F), we obtain a 83.3% preparation success rate within one timestep and 97.1% success rate within in two timesteps. In the future, these parameters can be adjusted based on maximal capability of caterpillar generation.

V. THE MEMTREE COMPILATION FRAMEWORK

A. Hierarchical Generation of Target State

In this section, we introduce our compilation scheme, which generates the target graph state from a set of primitive caterpillar states. We adapt a hierarchical generation method in which the generation process is modeled in a balanced binary tree (BBT), as shown in Fig. 6(a). The process starts from the leaves of BBT, which are linear graphs of logical qubits in our tree-encoding. Each pair of graph states is combined through fusions, forming a larger graph state as their parent. All fusions in the same layer are simultaneously operated on in one time step, and these time steps are performed sequentially from the layer of leaves (linear graphs) to the root (target graph state).

Here, we explain the reason for designing this hierarchical generation method. An alternative straightforward method is to apply all the fusions in one time step and directly generate the target graph from linear subgraphs. Considering the errors of fusion, this straightforward method is not practical: For example, even with an extremely high fusion success rate S_{fusion} (assuming $S_{fusion} = 0.99$), generating the target state of the 100-qubit VQE program requires $k > 1000$ times of fusion, leading to a success rate $S_{fusion}^k \sim 1e^{-5}$. On the contrary, in our hierarchical generation method, if any fusion

operation – as one node in the BBT fails, we only need to recover a sub-tree with that node as the root, as shown in Fig. 6 (a).

In our hierarchical generation method, the upper bound of generation overhead depends on a *critical path*. The *Critical path* is a path from a leaf to the root in BBT, which has the maximal total number of fusions along the path. The algorithm we introduce in the next subsection aims to reduce the critical path overhead. Overall, we use the balanced binary tree (BBT) to achieve a tradeoff between the overall success rate and execution time.

B. Building the Generation Tree of Target State

Here, we describe the details of our algorithm for building the balanced binary tree (BBT) of target state generation (Fig. 6(b)). Our algorithm is composed of two parts: (1) Dividing the target graph state into linear subgraphs, with a minimal number of total fusion operations (yellow box (I)). (2) Building the BBT while reducing the overhead on the critical path as much as possible (yellow box (II)-(III)).

1) *Dividing Target State*: We utilize the mix-integer-programming (MIP) solver in Gurobi to solve this problem. First, we model the program graph state as an undirected graph g_{prog} , with each qubit as a vertex $v \in V$, and each CZ-entanglement between qubits as an edge $e(i, j) \in E$. Then, we model the divided subgraphs as $g^l \subseteq g_{prog}$, and define specific constraints to ensure they are linear graphs. Next, we set the objective as the number of fusions to combine all the g^l into g_{prog} , to find its minimal value. We list the parameters, the constraints, and the objective function in the MIP model.

In the MIP model, we set each $e_{(v_1, v_2)}$ (CZ-entanglement) as a binary variable, with its value indicating whether it is cut or preserved:

$$x_{e, v_1, v_2} = \begin{cases} 1, & \text{if } e \text{ preserved in subgraphs} \\ 0, & \text{if } e \text{ cut for fusion.} \end{cases}, \forall e_{(v_1, v_2)} \in E$$

We add the constraint to ensure the subgraphs are linear – each vertex v should have its degree $deg(v) \leq 2$:

$$\sum_{v \in \{v_1, v_2\}} \forall x_{e, v_1, v_2} \leq 2, \quad \forall v \in V \quad (2)$$

The model's objective function is the total number of edge cuttings:

$$K = |E| - \sum_{e_{(v_1, v_2)} \in E} x_{e, v_1, v_2} \quad (3)$$

Overall, the MIP model can be formulated as

$$\begin{aligned} & \text{minimize objective } K \text{ (Eq. 3)} \\ & \text{s.t. constraint Eq. 2} \end{aligned}$$

Since the constraint Eq. 2 may lead to a cyclic linear graph, we apply post-processing on the subgraphs g^l , cutting one of its edges to make it acyclic. Furthermore, we cut these g^l into smaller linear subgraphs to comply with the maximum length of the caterpillar graph allowed in the specific hardware configuration. Finally, we obtain a set of linear subgraphs $G^l = \{g^l\}$ that can be resembled into g_{prog} .

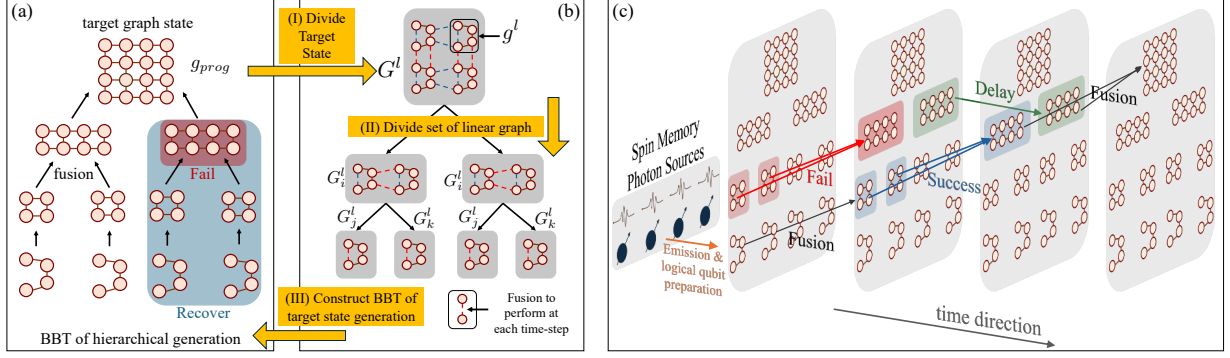


Fig. 6. Details of our MemTree compiler. (a) The hierarchical generation of target state based on BBT. (b) Our compiler framework for building BBT. (c) The overall pipeline for target state generation, from a time direction prospective. Each slice corresponds to a time step in the cycles.

2) *Constructing BBT of Target State Generation:* We construct the BBT by growing its layers hierarchically from the root G^l . Each node of the BBT is a subgraph of g_{prog} , and this subgraph is composed of a set of linear graphs G_i^l , which are a subset of G^l . During the construction, the subset G_i^l in each node is divided into two smaller subsets G_j^l and G_k^l , which are grown as the children of G_i^l .

For maximally reducing the overhead of the *critical path*, we adopt a straightforward but effective strategy: Starting from the root G^l , we search for each division from G_i^l to G_j^l, G_k^l with the minimal number of edges to cut. This strategy ensures that the division in the lower layers (closer to the root) has fewer fusions, as it is more likely to be involved in the *critical path*. In the meantime, we retain the balance of BBT to minimize its tree-height: when we divide G_i^l into G_j^l, G_k^l , the difference in cardinality between the sets G_j^l and G_k^l should not exceed a certain value. Based on the above strategies, we define another MIP model for dividing each G_i^l .

For each linear subgraph g^l that $g^l \in G_i^l$, we define a variable to determine whether it is divided into G_j^l or G_k^l :

$$y_{g^l} = \begin{cases} 1, & \text{if } g^l \text{ divided to } G_j^l \\ 0, & \text{if } g^l \text{ divided to } G_k^l \end{cases}, \forall g^l \in G_i^l$$

The following constraints are used to restrict the difference in cardinality between G_j^l and G_k^l :

$$|G_j^l| \geq 2^{\lfloor \log_2(|G_i^l|) \rfloor} \quad (4)$$

$$|G_k^l| \geq 2^{\lfloor \log_2(|G_i^l|) \rfloor} \quad (5)$$

The objective function for this MIP model is the number of edge cuts needed to divide G_i^l into G_j^l and G_k^l :

$$L = \sum_{v_1 \in g_1^l \wedge v_2 \in g_2^l, \forall v_1, v_2} |y_{g_1^l} - y_{g_2^l}|, \quad \forall e_{(v_1, v_2)} \in E \quad (6)$$

Overall, the MIP model can be formulated as

minimize objective L (Eq. 6)

s.t. constraint Eqs. (4, 5)

The node dividing process runs recursively until it reaches the leaf node where $G_j^l = G_k^l = 1$. Finally, we can build our BBT of the generation process: the edges cut from G_i^l to G_j^l, G_k^l in each layer represents the number of fusion operations that need to be performed at each time step.

These two MIP models described above have $O(|E|)$ and $O(|G_i^l|)$ complexities in terms of the number of binary variables, which allows for a relatively low compilation runtime. In Sec. VII we evaluate the overall runtime of our algorithms.

C. Pipeline for Target State Generation

With the BBT of generation described above, in Fig. 6(c) we illustrate the generation process from a time-directional perspective. Similar to the time-like model in OneAdapt [74], the photon source iteratively generates caterpillar states, forming a pipeline for generating target states. First, the emitted caterpillar states are prepared into the linear graphs of tree-encoded logical qubits. Then, at each time step, each layer of subgraphs performs fusion operations to merge into their parent subgraphs while being forwarded in the pipeline. When the fusion into a subgraph fails (Fig. 6(c) red arrows), its sibling subgraph is delayed to the next time step (green arrow) and waits for the next successful generation of this subgraph (blue arrows). The descendants of this sibling subgraph are also delayed. Overall, we aim to generate as many target states as possible, thereby maximizing the execution shots of the quantum program within the limited time cycles of the pipeline.

VI. EXPERIMENTAL METHODOLOGY

A. Baselines

1) *Boosted Fusion Schemes:* In the first part, we select baselines within the same architecture – quantum spin memory. We evaluate our tree-encoded fusion scheme through comparison with two mainstream boosted-fusion schemes introduced in Sec. IV.A: the **redundantly-encoded fusion** [25] and **repeat-until-success (RUS) fusion** [21], [40]. We implement their up-to-date protocols according to the most recent research [12] and integrate them into our compiler framework. Based on the analysis and simulation from the

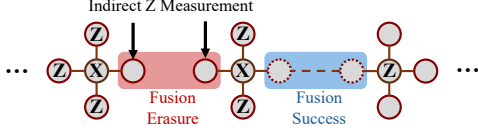


Fig. 7. Addressing erasure error in OneAdapt-ET.

papers [12], [25] of these schemes, we choose the code sizes $m_{Redun} = 5$ and $m_{RUS} = 6$ for optimal error-tolerance performance.

2) *SOTA MBQC Compiler*: In the second part, we select the baseline from other MBQC architectures, namely the all-photononic and emitter-based architectures.

We compare our framework with the SOTA compiler of the all-photononic architecture – **OneAdapt** [74]. Furthermore, we improve OneAdapt by designing an erasure-tolerance scheme and integrating the scheme into it, namely **OneAdapt-ET**. OneAdapt-ET addresses the qubit that undergoes fusion erasure by applying an indirect Z-measurement based on the graph state property introduced in Sec. IV.B. Specifically, we apply an X-measurement on a neighboring free qubit that is not involved in the normalization path, then apply a Z-measurement on all other adjacent qubits of this neighboring free qubit. The scheme of OneAdapt-ET is depicted in Fig. 7.

For a fair comparison, we set the configuration of the *time-like edge length limit* D_f in OneAdapt and OneAdapt-ET as $D_f = 30$ virtual layers, strictly following the evaluation settings in [74]. A recent experimental demonstration [52] from PsiQuantum claims a 125 MHz photon source pumping rate in their all-photononic architecture. While in OneAdapt, each virtual layer includes 4 physical photon resource layers ($PL = 4$), and the maximal duration of the time-like edge is 960 ns. Correspondingly, we set our maximal delay to 32 emission layers, since the maximal emission time of each caterpillar state layer is 30 ns, according to the experiments in [25]. Additionally, we set the resource state layer (RSL) of OneAdapt/OneAdapt-ET as $14n \times 14n$ 2D size, according to the source code of OneAdapt.

We select the SOTA compilation framework **RLGS** [38] for the emitter-based architecture. Although the emitter-CZ operation is still out of reach in real experiments, we can still compare it as a long-term future architecture. Due to the distinctive hardware of emitter-based architecture, RLGS uses a set of different metrics [38]. Hence, we compare with RLGS specifically on fidelity metrics reported in their paper: (1) Fidelity affected by decoherence error (F_{de}), and (2) Fidelity affected by emitter-CZ (F_{CZ}), which corresponds to the fidelity affected by fusion (F_{fus}) in our framework.

B. Benchmark Programs

We select a set of benchmark programs, including the Bernstein–Vazirani algorithm (BV), the Quantum Approximate Optimization Algorithm (QAOA), Grover’s Algorithm (Grover), the Quantum Fourier transform (QFT), quantum Hamiltonian simulation (QSIM), the Ripple Carry Adder (RCA), and the

Variational Quantum Eigensolver (VQE). In the comparison with redundantly-encoded and RUS fusion schemes, we set the size of the benchmark program from 2-qubits to 20-qubits. This is because these two baselines of fusion schemes have a prolonged execution time, which is out of reach in simulation. For the comparison between our compiler and OneAdapt [74], we use exactly the same benchmark programs and settings, with program sizes of 36, 64, and 100-qubits.

C. Noise Model

1) *Fusion Failure and Erasure Errors*: Here are the details of our simulator for spin memory architecture PQC. Based on recent experimental works on spin memory architecture [30], [43] and linear-optical PQC [3], [6], [52], we simulate the following important errors in PQC: fusion failure and erasure errors, photon source decoherence, and fusion infidelity (indistinguishability). We set $1 - p_{fail} = 0.75$ as the fusion success rate when assuming no erasure error, which corresponds to the error model introduced in Sec. 5.1 of the OneAdapt paper [74]. This success rate can be achieved by utilizing additional interferometric setups reported in previous works [18], [22], [49].

2) *Decoherence Errors*: We simulate the photon source (emitter) decoherence based on $F_{de} = e^{-\frac{N_q t_{gen}}{T_2}}$, in accordance with the error model used in RLGS [38]. We set the dephasing time of RLGS based at $T_2 = 4.4 \mu s$, as reported by [31], [38]. As for OneAdapt and MemTree, we estimate the dephasing time based on the Bell state (GHZ state) fidelity reported in corresponding hardware demonstration [30], [52]. In [52] the fidelity of a 2-qubit Bell state is 99.22% for all-photononic architecture, while [30] reports a 95% optimal fidelity of a 4-qubit GHZ state for spin memory architecture. The dephasing time T_2 can be calculated by

$$T_2 = \frac{-N_q t_{gen}}{\ln(F_{state})}, \quad N_q = \text{\#qubit}, \quad t_{gen} = \text{generation time}.$$

The dephasing time for each architecture is listed in Table. II.

3) *Coherent Errors of Fusion Operation*: We simulate the overall fusion fidelity $F_{fus} = \sigma_{fus}^{N_{fus}}$, corresponding with $F_{CZ} = \sigma_{CZ}^{N_{CZ}}$ reported in RLGS [38]. We set the fidelity of each emitter-CZ operation at $\sigma_{CZ} = 99\%$ for RLGS, as reported by [57] in the form of pulse-level simulation result. Based on the *Hong-Ou-Mandel (HOM)* visibility $V_{HOM} = 99.5\%$ reported in [52], we set the fidelity of type-II fusion operation at $\sigma_{fus} = \frac{1+V_{HOM}}{2} = 99.75\%$ based on [27]. Meanwhile, we set the OSRP fidelity for spin memory at 99%, as reported in [30].

TABLE II
DETAILS OF NOISE MODEL WE ADAPT ON THE FIDELITY COMPARISON.

Compiler	OneAdapt	RLGS	MemTree
Based on Platform	PsiQuantum	[57] (Simulation)	Quandela
Dephasing T_2	2.04 μs	4.4 μs	2.34 μs
CZ (Fusion) Fidelity	99.75%	99%	99%
t_{cycle}	8 ns	10 ns (emitter-CZ)	30 ns

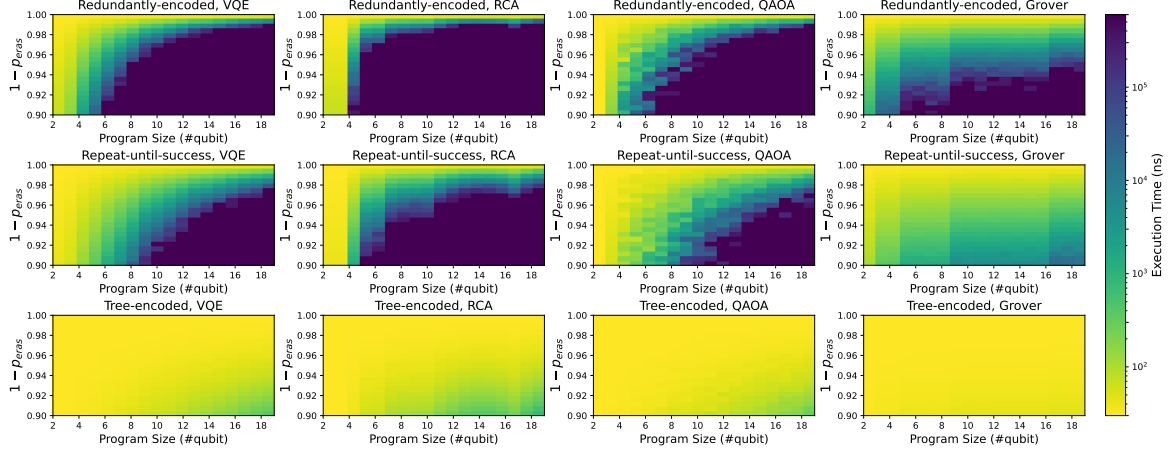


Fig. 8. Execution time comparison between tree-encoded scheme and baselines.

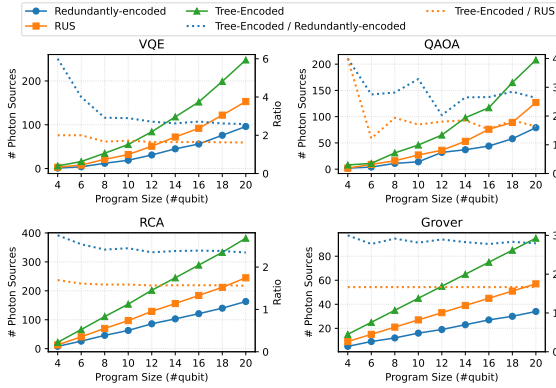


Fig. 9. Number of required photon sources comparison between tree-encoded scheme and baselines.

D. MemTree Simulator Configurations

We simulate the generation of caterpillar states according to hardware configurations reported in [30], [43]. Specifically, each qubit in a caterpillar state is emitted through an excitation pulse of InGaAs semiconductor quantum-dots, while assisted by an optical spin rotation pulse (OSRP) to define the caterpillar structure [30]. Generation of a caterpillar graph state includes a 12 ns initialization time, plus a 0.6 ns time cycle for the emission of each qubit. The near-term spin memory technique can produce a caterpillar state with at most 30-qubit [30], which is set as the maximal size of the caterpillar in our framework. For calculating the average execution time, we simulate 2×10^4 cycles of caterpillar state emissions and divide the total time by the number of successful shots executed during these cycles. In addition, we choose $b = 4$ as the tree-encoding parameter, based on the parametric study in Sec VII.D.

E. Metrics

We evaluate the performance of our compiler using the following metrics: average execution time of quantum programs,

number of photon sources, compilation runtime, and fidelity of the quantum program. For fidelity, we include decoherence fidelity F_{de} , and CZ (fusion) fidelity F_{CZ} (F_{fus}).

VII. EVALUATION

A. Comparison with Boosted-Fusion Schemes

Fig. 8 and Fig. 9 present the comparison of our tree-encoded fusion scheme with the redundantly-encoded and RUS fusion schemes under the hardware configurations of the quantum spin memory architecture. In this comparison, all fusion schemes are integrated in MemTree with the same compilation algorithm. While fixing the fusion failure rate $p_{fail} = 0.25$ (thus $1 - p_{fail} = 0.75$), we compare the program execution time and the number of required photon sources. The program size (#qubit) varies from 2-qubit to 20-qubit, and the erasure rate during fusion (p_{eras}) varies from 0% to 10%. Due to the extremely large simulation overhead when the program size scales up, we truncate the execution time to at most 6×10^5 ns. Fig. 8 shows that our scheme significantly reduces the average execution time of quantum programs, gaining an average reduction rate of 1.9×10^{-3} and 1.7×10^{-2} , compared to redundantly-encoded and RUS, respectively. Fig. 9 shows that our scheme consumes more photon sources than the baseline schemes, with an average of $2.55\times$ and $1.63\times$ compared to redundantly-encoded and RUS, respectively. Nevertheless, considering the exponential reduction in execution time, we believe that the tree-encoded scheme is an appropriate strategy for *trading space for time*. Besides, it can be observed that for tree-encoded fusion, its disadvantage on photon sources decreases as the #qubit grows (Fig. 9 dotted lines).

B. Comparison with SOTA Compilers of Other Architectures

Execution Time. Fig. 10(a)-(c) present the comparison of our framework MemTree with OneAdapt and OneAdapt-ET on average execution time, the number of photon sources, and compilation runtime. In Fig. 10(a), the execution time results on benchmarks with 36, 64, and 100-qubits are shown, with

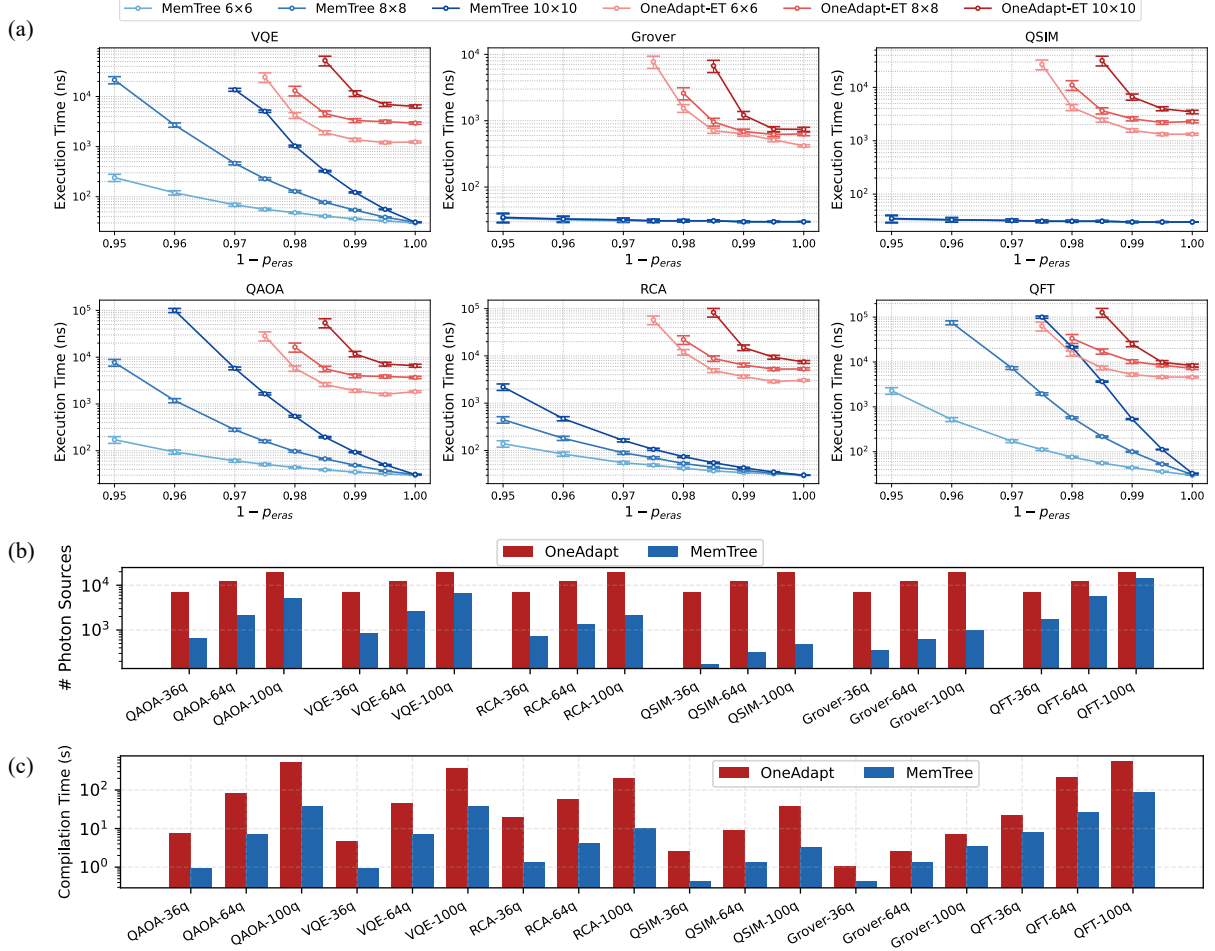


Fig. 10. Comparison of MemTree with OneAdapt [74] and OneAdapt-ET. (a) The average execution time of quantum programs, when $p_{eras} = 0$, the results are evaluated on OneAdapt without erasure-tolerance strategy. The error bars represent the value range with a statistical 95% CI (confidence interval), over 1000 times of experiment and each with 2×10^4 shots. (b) Number of required photon sources. (c) Total compilation runtime of compilers.

varying fusion erasure rates p_{eras} from 0% to 5%. Note that the realistic p_{eras} estimated from the hardware experiment is on the order of $\sim 1\%$ [52]. We set a simulation limit for the execution time (2×10^5 ns), since a longer execution time requires > 80 hours of simulation on our machine. The results show that our compiler framework achieves an exponential improvement in execution time, and the reduction rates are 1.5×10^{-2} , 1.1×10^{-2} , 3.8×10^{-2} , 5.6×10^{-3} , 1.1×10^{-2} , and 8.8×10^{-3} for VQE, QAOA, Grover, RCA, QSIM, and QFT, respectively. Shown in Fig. 10(b)-(c), the number of photon sources is reduced to $0.18\times$ on average, and the compilation time is reduced $0.14\times$ on average compared to OneAdapt. For the Grover and QSIM benchmarks, the intrinsic structure of their graph state leads to a relatively low number of fusions when divided into caterpillar states, and this number does not scale with program size, while larger programs only require more photon sources.

Circuit Fidelity. Fig. 11 presents the comparison of MemTree with RLGS [38] on F_{de} and F_{CZ}/F_{fus} , using the

benchmark results (QFT, QAOA, BV) reported in their paper. Note that the fusion operation in our architecture behaves similarly to emitter-CZ in the emitter-based architecture of RLGS, thus we compare our F_{fus} with their F_{CZ} . The results show that we achieved a significant improvement on F_{de} , especially an exponential enhancement for QFT and QAOA. The results show that MemTree outperforms OneAdapt and RLGS both in F_{de} and F_{CZ} , and the advantage grows with #qubit.

C. Ablation Study

We conduct the following ablation study to verify that our performance gains over OneAdapt primarily stem from our novel tree-encoded fusion rather than merely from differences in PQC architecture or hardware configurations. Here, we compare three different compiler settings: MemTree with the RUS fusion scheme (the same we used in Sec. VII.A), MemTree, and OneAdapt-ET, on 36-qubit benchmarks with $p_{eras} = 0.5\%$. The results are shown in Fig. 12(b). It can be observed that when the Tree-encoded fusion is replaced by

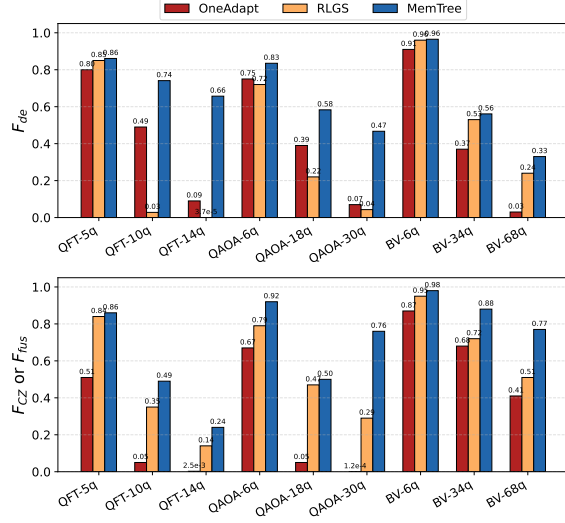


Fig. 11. Comparison on decoherence errors and CZ errors between OneAdapt [74], RLGS [38] and MemTree.

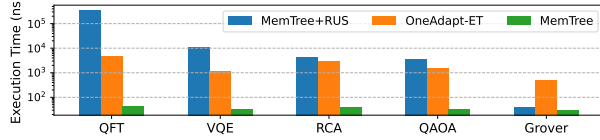


Fig. 12. Encoding parameter study and ablation study.

the RUS fusion scheme, it under-performs OneAdapt on all benchmarks except Grover. This ablation experiment supports for the novelty and effectiveness of our design of tree-encoded fusion scheme.

D. Trade-Off Analysis of System Characterizations

In Table. III we analyze the system characterization of each compiler with the following metrics: *emission frequency* = $\#(\text{physical photon}) / \text{ns}$, *CZ frequency* = $\#(\text{CZ or fusion operation}) / \text{ns}$, and *utilization rate* = $\#(\text{logical qubit}) / \#(\text{total physical photon})$. From the data we can find MemTree establishing a proper trade-off between operation frequency and photon utilization rate. (i) Higher frequency (OneAdapt) leads to shorter QPU runtime, but requires larger number of noisy operations, hence the dominant errors derive from fusion operations. (ii) Higher utilization rate (RLGS) is based on fewer and more reliable CZ operations, but leads to lower frequency and longer QPU runtime, hence the dominant errors is decoherence. (iii) Based on the spin memory architecture and tree-encoded scheme, MemTree is designed to reach a trade-off between frequency and logical CZ (fusion) reliability. This prevents extremely high error rate deriving from CZ (fusion) or decoherence, thus MemTree outperforms both baselines in the evaluation of Fig.11.

TABLE III
FREQUENCY-NOISE ANALYSIS (ON 30-QUBIT QAOA)

Compiler	OneAdapt	MemTree	RLGS
Emission Frequency (/ns)	$\approx 2 \times 10^3$	$\approx 7 \times 10^2$	≈ 10
CZ Frequency (/ns)	$\approx 1 \times 10^3$	$\approx 2 \times 10^2$	≈ 1
Utilization rate	$\approx 0.03\%$	$\approx 10\%$	100%
Dominant Error	Fusion	F-D Tradeoff	Decoherence

E. Feed-Forward Control in PQC System

There are two cases in MemTree where feed-forward control is required. (i) In the tree-encoded fusion scheme, the measurement basis of ancillary qubits is updated according to the fusion outcome to handle fusion failure and erasure. This feed-forward is not on the critical optical path: once a fusion outcome is detected, the affected qubits can remain as dangling qubits, and the controller only needs to record which recovery pattern will later be applied. Therefore, the corrective measurements do not need to be triggered immediately after fusion; they only need to be synchronized before the dangling branch is consumed by later graph-state measurements, or before the final measurement stage of the quantum program, following the adaptive-measurement model of MBQC [68].

(ii) In the graph-generation pipeline (Sec. V-C), feed-forward is also needed to decide whether a sub-graph should be delayed to the next timestep after an unsuccessful fusion. This control path consists of photon detection, a small classical decision circuit, and the timing/delay module. In our target hardware, the measurement signal is produced by *superconducting nanowire single-photon detectors* with latency below 50 ps [44]; the detector outputs are then passed to a small combinational logic block (e.g., a b -input AND/OR network for the b fusion branches), which decides whether the logical fusion succeeds or whether the sibling sub-graph must be stalled. The resulting control signal drives the time-delay module. We estimate the total classical feed-forward latency to be below 5 ns, and implement this logic using `FFCircuitProvider` in Perceval [24]. Since this latency is well below one emission timestep in spin-memory hardware, the updated measurement pattern and sub-graph schedule can be synchronized before the next emission layer begins.

F. Real Photonic Hardware Experiment

Here we perform a small-scale experiment on real photonic quantum hardware, specifically the Quandela Benlenos QPU [2]. In this experiment, the optical hardware circuit is built with the Perceval PQC toolkit [24]. We illustrate the most important part of the hardware circuit in Fig. 13, which is the fusion operation and dealing with possible fusion failure or

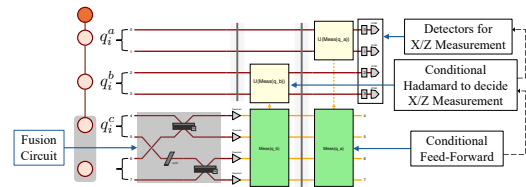


Fig. 13. The optical hardware circuit for tree-encoded fusion.

erasure. In the circuit, each qubit is represented by a dual-rail encoding – two photon modes (e.g., H or V polarization) are used to encode one qubit. As in Fig. 13, the fusion circuit is a permutation of photon modes from the two qubits, followed by a phase shift and two beam splitters. Corresponding to the tree-encoded scheme in Fig. 4(b), the fusion outcome from q_i^c is detected and triggers a conditional feed-forward operation on q_i^a and q_i^b . The feed-forward operation decides whether to apply an X or Z measurement on q_i^a and q_i^b , complying with the error-tolerant measurement patterns. The characterization of photonic hardware are as follows: HOM indistinguishability = 92.0%, transmittance = 5.16%, $g_2 = 2.0\%$.

We compile QAOA programs (6–12 qubits) using MemTree, and execute them on photonic hardware. In Fig. 14, MemTree are compared with repeat-until-success (RUS) scheme [21] executed on photonic hardware, and Qiskit transpilation [32] executed on IBM Torino superconducting quantum computer. We use the *EfficientSU2* (SU2) ansatz for QAOA as the default settings, and add a setting of *RealAmplitudes* (RA) ansatz to MemTree to extend the comparison. Note that in RA ansatz the parameterized rotation gates are restricted to $R_Y(\theta)$ only, as a simplified ansatz. The results are evaluated in two metrics, which are *Probability of Successful Trial* (PST) [15], [42], [47], [62], [63] and *Inference Strength* (IST) [42], [50], [62]. From the evaluation results in Fig. 14, on average, MemTree (SU2 ansatz) achieves an improvement on Probability of Success Trial (PST) by the ratio of $2.68\times$ compared to [RUS + photonic] and $2.20\times$ compared to [Qiskit + superconducting]. Also, MemTree achieves an improvement on Inference Strength (IST) by the ratio of $3.23\times$ compared to [RUS + photonic] and $2.91\times$ compared to [Qiskit + superconducting].

From above results, we analyze the reason that PQC hardware outperforms superconducting QPU: PQC has significantly lower crosstalk than matter-based systems, while spin-memory single photon sources are isolated and have no interaction with each others. Consequently, PQC provides higher parallelism of CZ operations, and reduce circuit execution time. As we protect the fusion (CZ) with tree-encode scheme, the overall quantum noise is efficiently suppressed. Generally, SU2 ansatz performs better than RA ansatz for MemTree, however RA starts to outperform SU2 when the number of qubits scales up. This attributes to the fewer number of parameters in RA ansatz, which leads to lower complexity of optimization when #qubit scales up.

VIII. RELATED WORKS AND DISCUSSIONS

Recent research on quantum computer systems has primarily addressed error correction for superconducting platforms [4], [14], [64], [69] via compilation advances [16], [60] and other architectural improvements [45], [61], [67], [70]. For photonic quantum computing, compilation frameworks target measurement-based systems [76], probabilistic fusion operations [73], and bosonic encodings [77]. FCM [46] uses wire cutting to partition circuits and reduce fusion counts through classical post-processing, while our work addresses fusion erasure errors through tree-encoded fusion schemes in spin

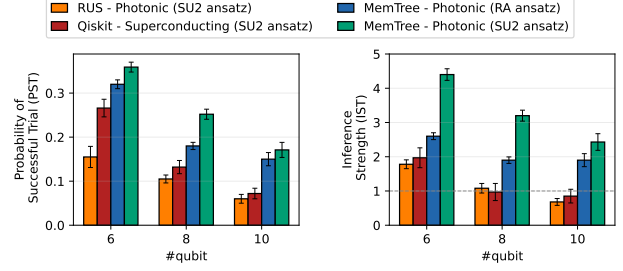


Fig. 14. Comparing the performance of QAOA programs on real hardware between superconducting qubits and photonic spin memory. The error bars stand for the standard error (SE).

memory architecture. FMCC [39] reduces photonic MBQC cluster-state depth by exploiting flexible mapping variants with dynamic programming and heuristics.

Prior work on biased-noise QEC, such as the XZZX surface code [8] and superconducting dual-rail cavity codes [65], studies circuit-model protection by tailoring syndrome-based correction to a hardware-specific error hierarchy. By contrast, our setting is fusion-based MBQC on optical photonic graph states, where the dominant challenge arises from imperfect graph-state generation itself. In particular, fusion *failure* and fusion *erasure* describe two distinct error modes of the fusion primitive, but they do not form a biased-noise model in the usual QEC sense, since neither is simply a dominant variant of the other. Our method corrects graph-generation uncertainty through graph-state measurement patterns and indirect measurements on ancillary qubits, rather than through circuit-level decoding of a biased code.

Beyond spin-memory hardware, the same loss-tolerant logical-fusion idea can also be applied to other PQC architectures whenever graph states are built through fusion. In particular, all-photonic schemes such as fusion-based quantum computation and OneAdapt already rely on small resource states and repeated fusion measurements [5], [74]. In that case, our method can be adapted by replacing the original fusion units with tree-encoded logical qubits, while changing only the resource-state preparation procedure: spin-memory hardware prepares them efficiently from caterpillar states, whereas all-photonic systems would synthesize them from Bell pairs or other small photonic resource states before entering the fusion pipeline. The loss-tolerant recovery mechanism itself remains unchanged, since it still follows graph-state measurement rules and indirect measurements [68].

IX. CONCLUSION

In this work, we present MemTree based on spin-memory architecture, while introducing the tree-encoded fusion to address fusion erasure. It leads to substantial reduction on execution time and improvement on fidelity, while maintaining a reasonable photon-resource overhead, outperforming SOTA PQC compilers. Moreover, the experiment on real hardware unleash the potential advantages of PQC, when erasure errors are properly addressed.

REFERENCES

- [1] "Tree cluster-state code," in *The Error Correction Zoo*, V. V. Albert and P. Faist, Eds., 2024. [Online]. Available: https://errorcorrectionzoo.org/c/tree_cluster
- [2] "High performance computing, artificial intelligence, and quantum computing convergence," 2025. [Online]. Available: <https://www.quandela.com/wp-content/uploads/2025/10/202510-Quantum-HPC-Quandela-Whitepaper.pdf>
- [3] H. Aghaee Rad, T. Ainsworth, R. N. Alexander, B. Altieri, M. F. Askarani, R. Baby, L. Banchi, B. Q. Baragiola, J. E. Bourassa, R. S. Chadwick, I. Charania, H. Chen, M. J. Collins, P. Contu, N. D'Arcy, G. Dauphinais, R. De Prins, D. Deschenes, I. Di Luch, S. Duque, P. Edke, S. E. Fayer, S. Ferracin, H. Ferretti, J. Gefaell, S. Glancy, C. González-Arciniegas, T. Grainge, Z. Han, J. Hastrup, L. G. Helt, T. Hillmann, J. Hundal, S. Izumi, T. Jaeken, M. Jonas, S. Kocsis, I. Krasnokutska, M. V. Larsen, P. Laskowski, F. Laudenbach, J. Lavoie, M. Li, E. Lomonte, C. E. Lopetegui, B. Luey, A. P. Lund, C. Ma, L. S. Madsen, D. H. Mahler, L. Mantilla Calderón, M. Menotti, F. M. Miatto, B. Morrison, P. J. Nadkarni, N. Nakamura, L. Neuhaus, Z. Niu, R. Noro, K. Papirov, A. Pesah, D. S. Phillips, W. N. Plick, T. Rogalsky, F. Rortais, J. Sabines-Chesterking, S. Safavi-Bayat, E. Sazhaev, M. Seymour, K. Rezaei Shad, M. Silverman, S. A. Srinivasan, M. Stephan, Q. Y. Tang, J. F. Tasker, Y. S. Teo, R. B. Then, J. E. Tremblay, I. Tzitrin, V. D. Vaidya, M. Vasmer, Z. Vernon, L. F. S. M. Villalobos, B. W. Walshe, R. Weil, X. Xin, X. Yan, Y. Yao, M. Zamani Abnili, and Y. Zhang, "Scaling and networking a modular photonic quantum computer," *Nature*, vol. 638, no. 8052, pp. 912–919, Feb. 2025. [Online]. Available: <https://www.nature.com/articles/s41586-024-08406-9>
- [4] R. Ayanzadeh, N. Alavisamani, P. Das, and M. Qureshi, "Frozenqubits: Boosting fidelity of qaoa by skipping hotspot nodes," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 2023, pp. 311–324.
- [5] S. Bartolucci, P. Birchall, H. Bombin, H. Cable, C. Dawson, M. Gimeno-Segovia, E. Johnston, K. Kieling, N. Nickerson, M. Pant, F. Pastawski, T. Rudolph, and C. Sparrow, "Fusion-based quantum computation," *Nature Communications*, vol. 14, no. 1, p. 912, 2023.
- [6] S. Bartolucci, P. Birchall, H. Bombin, H. Cable, C. Dawson, M. Gimeno-Segovia, E. Johnston, K. Kieling, N. Nickerson, M. Pant, F. Pastawski, T. Rudolph, and C. Sparrow, "Fusion-based quantum computation," *Nature Communications*, vol. 14, no. 1, p. 912, Feb. 2023, publisher: Nature Publishing Group. [Online]. Available: <https://www.nature.com/articles/s41467-023-36493-1>
- [7] T. J. Bell, L. A. Pettersson, and S. Paesani, "Optimizing Graph Codes for Measurement-Based Loss Tolerance," *PRX Quantum*, vol. 4, no. 2, p. 020328, May 2023. [Online]. Available: <https://link.aps.org/doi/10.1103/PRXQuantum.4.020328>
- [8] J. P. Bonilla Ataides, D. K. Tuckett, S. D. Bartlett, S. T. Flammia, and B. J. Brown, "The xxxz surface code," *Nature Communications*, vol. 12, no. 1, p. 2172, 2021.
- [9] J. Borregaard, H. Pichler, T. Schröder, M. D. Lukin, P. Lodahl, and A. S. Sørensen, "One-Way Quantum Repeater Based on Near-Deterministic Photon-Emitter Interfaces," *Physical Review X*, vol. 10, no. 2, p. 021071, Jun. 2020. [Online]. Available: <https://link.aps.org/doi/10.1103/PhysRevX.10.021071>
- [10] H. J. Briegel, D. E. Browne, W. Dür, R. Raussendorf, and M. Van den Nest, "Measurement-based quantum computation," *Nature Physics*, vol. 5, no. 1, pp. 19–26, 2009.
- [11] D. E. Browne and T. Rudolph, "Resource-efficient linear optical quantum computation," *Physical Review Letters*, vol. 95, no. 1, p. 010501, 2005. [Online]. Available: <https://journals.aps.org/prl/abstract/10.1103/PhysRevLett.95.010501>
- [12] M. L. Chan, T. J. Bell, L. A. Pettersson, S. X. Chen, P. Yard, A. S. Sørensen, and S. Paesani, "Tailoring fusion-based photonic quantum computing schemes to quantum emitters," *PRX Quantum*, vol. 6, no. 2, p. 020304, Apr. 2025, arXiv:2410.06784 [quant-ph]. [Online]. Available: <http://arxiv.org/abs/2410.06784>
- [13] D. Cogan, Z.-E. Su, O. Kenneth, and D. Gershoni, "Deterministic generation of indistinguishable photons in a cluster state," *Nature Photonics*, vol. 17, no. 4, pp. 324–329, Apr. 2023, publisher: Nature Publishing Group. [Online]. Available: <https://www.nature.com/articles/s41566-022-01152-2>
- [14] P. Das, C. A. Pattison, S. Manne, D. M. Carmean, K. M. Svore, M. Qureshi, and N. Delfosse, "Afs: Accurate, fast, and scalable error-decoding for fault-tolerant quantum computers," in *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2022, pp. 259–273.
- [15] P. Das, S. S. Tannu, P. J. Nair, and M. Qureshi, "A case for multi-programming quantum computers," in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, 2019, pp. 291–303.
- [16] Y. Ding, P. Gokhale, S. F. Lin, R. Rines, T. Propson, and F. T. Chong, "Systematic crosstalk mitigation for superconducting qubits via frequency-aware compilation," in *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2020, pp. 201–214.
- [17] S. E. Economou, N. Lindner, and T. Rudolph, "Optically generated 2-dimensional photonic cluster state from coupled quantum dots," *Physical review letters*, vol. 105, no. 9, p. 093601, 2010. [Online]. Available: <https://journals.aps.org/prl/abstract/10.1103/PhysRevLett.105.093601>
- [18] F. Ewert and P. van Loock, "3/4-efficient bell measurement with passive linear optics and unentangled ancillae," *Physical review letters*, vol. 113, no. 14, p. 140403, 2014.
- [19] M. Gimeno-Segovia, "Deterministic Generation of Large-Scale Entangled Photonic Cluster State from Interacting Solid State Emitters," *Physical Review Letters*, vol. 123, no. 7, 2019.
- [20] M. Gimeno-Segovia, T. Rudolph, and S. E. Economou, "Deterministic generation of large-scale entangled photonic cluster state from interacting solid state emitters," *Physical review letters*, vol. 123, no. 7, p. 070501, 2019. [Online]. Available: <https://journals.aps.org/prl/abstract/10.1103/PhysRevLett.123.070501>
- [21] G. d. Gliniasty, P. Hilaire, P.-E. Emeriau, S. C. Wein, A. Salavrakos, and S. Mansfield, "A Spin-Optical Quantum Computing Architecture," *Quantum*, vol. 8, p. 1423, Jul. 2024, publisher: Verein zur Förderung des Open Access Publizierens in den Quantenwissenschaften. [Online]. Available: <https://quantum-journal.org/papers/q-2024-07-24-1423/>
- [22] W. P. Grice, "Arbitrarily complete bell-state measurement using only linear optical elements," *Physical Review A—Atomic, Molecular, and Optical Physics*, vol. 84, no. 4, p. 042331, 2011.
- [23] M. Hein, W. Dür, J. Eisert, R. Raussendorf, M. Nest, and H.-J. Briegel, "Entanglement in graph states and its applications," *arXiv preprint quant-ph/0602096*, 2006. [Online]. Available: <https://arxiv.org/abs/quant-ph/0602096>
- [24] N. Heurtel, A. Fyrrillas, G. d. Gliniasty, R. Le Bihan, S. Malherbe, M. Pailhas, E. Bertasi, B. Bourdoncle, P.-E. Emeriau, R. Mezher, L. Music, N. Belabas, B. Valiron, P. Senellart, S. Mansfield, and J. Senellart, "Perceval: A Software Platform for Discrete Variable Photonic Quantum Computing," *Quantum*, vol. 7, p. 931, Feb. 2023. [Online]. Available: <https://doi.org/10.22331/q-2023-02-21-931>
- [25] P. Hilaire, L. Vidro, H. S. Eisenberg, and S. E. Economou, "Near-deterministic hybrid generation of arbitrary photonic graph states using a single quantum emitter and linear optics," *Quantum*, vol. 7, p. 992, Apr. 2023, publisher: Verein zur Förderung des Open Access Publizierens in den Quantenwissenschaften. [Online]. Available: <https://quantum-journal.org/papers/q-2023-04-27-992/>
- [26] —, "Near-deterministic hybrid generation of arbitrary photonic graph states using a single quantum emitter and linear optics," *Quantum*, vol. 7, p. 992, 2023. [Online]. Available: <https://quantum-journal.org/papers/q-2023-04-27-992/>
- [27] C.-K. Hong, Z.-Y. Ou, and L. Mandel, "Measurement of subpicosecond time intervals between two photons by interference," *Physical review letters*, vol. 59, no. 18, p. 2044, 1987.
- [28] P. Hoyer, M. Mhalla, and S. Perdrix, "Resources required for preparing graph states," in *International Symposium on Algorithms and Computation*. Springer, 2006, pp. 638–649. [Online]. Available: https://link.springer.com/chapter/10.1007/11940128_64
- [29] H. Huet, P. R. Ramesh, S. C. Wein, N. Coste, P. Hilaire, N. Somaschi, M. Morassi, A. Lemaître, I. Sagnes, M. F. Doty, O. Krebs, L. Lanco, D. A. Fioretto, and P. Senellart, "Deterministic and reconfigurable graph state generation with a single solid-state quantum emitter," *Nature Communications*, vol. 16, no. 1, p. 4337, May 2025, publisher: Nature Publishing Group. [Online]. Available: <https://www.nature.com/articles/s41467-025-59693-3>
- [30] H. Huet, P. Ramesh, S. Wein, N. Coste, P. Hilaire, N. Somaschi, M. Morassi, A. Lemaître, I. Sagnes, M. Doty et al., "Deterministic and reconfigurable graph state generation with a single solid-state quantum

- emitter,” *Nature communications*, vol. 16, no. 1, p. 4337, 2025. [Online]. Available: <https://www.nature.com/articles/s41467-025-59693-3>
- [31] L. Huthmacher, R. Stockill, E. Clarke, M. Hugues, C. Le Gall, and M. Atatüre, “Coherence of a dynamically decoupled quantum-dot hole spin,” *Physical Review B*, vol. 97, no. 24, p. 241413, 2018.
 - [32] A. Javadi-Abhari, M. Treinish, K. Krsulich, C. J. Wood, J. Lishman, J. Gacon, S. Martiel, P. D. Nation, L. S. Bishop, A. W. Cross, B. R. Johnson, and J. M. Gambetta, “Quantum computing with Qiskit,” 2024.
 - [33] E. Kaur, A. Patil, and S. Guha, “Resource-efficient loss-aware photonic graph state preparation using atomic emitters,” *arXiv preprint arXiv:2402.00731*, 2024. [Online]. Available: <https://arxiv.org/abs/2402.00731>
 - [34] P. G. Kwiat, K. Mattle, H. Weinfurter, A. Zeilinger, A. V. Sergienko, and Y. Shih, “New high-intensity source of polarization-entangled photon pairs,” *Physical Review Letters*, vol. 75, no. 24, p. 4337, 1995. [Online]. Available: <https://journals.aps.org/prl/abstract/10.1103/PhysRevLett.75.4337>
 - [35] S.-H. Lee and H. Jeong, “Graph-theoretical optimization of fusion-based graph state generation,” *Quantum*, vol. 7, p. 1212, 2023. [Online]. Available: <https://quantum-journal.org/papers/q-2023-12-20-1212/>
 - [36] B. Li, S. E. Economou, and E. Barnes, “Photonic resource state generation from a minimal number of quantum emitters,” *npj Quantum Information*, vol. 8, no. 1, p. 11, 2022. [Online]. Available: <https://www.nature.com/articles/s41534-022-00522-6>
 - [37] Y. Li, P. C. Humphreys, G. J. Mendoza, and S. C. Benjamin, “Resource costs for fault-tolerant linear optical quantum computing,” *Physical Review X*, vol. 5, no. 4, p. 041007, 2015. [Online]. Available: <https://journals.aps.org/prx/abstract/10.1103/PhysRevX.5.041007>
 - [38] Y. Li, Y. Dai, A. Pawar, R. Dong, J. Yang, Y. Zhang, and X. Tang, “Reinforcement Learning-Guided Graph State Generation in Photonic Quantum Computers,” in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, ser. ISCA ’25. New York, NY, USA: Association for Computing Machinery, 2025, pp. 1598–1612. [Online]. Available: <https://dl.acm.org/doi/10.1145/3695053.3731085>
 - [39] Y. Li, A. Pawar, Z. Mo, Y. Zhang, J. Yang, and X. Tang, “Fmcc: Flexible measurement-based quantum computation over cluster state,” in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 4*, 2024, pp. 111–126.
 - [40] Y. L. Lim, “Repeat-Until-Success Linear Optics Distributed Quantum Computing,” *Physical Review Letters*, vol. 95, no. 3, 2005. [Online]. Available: <https://journals.aps.org/prl/abstract/10.1103/PhysRevLett.95.030505>
 - [41] N. H. Lindner and T. Rudolph, “Proposal for pulsed on-demand sources of photonic cluster state strings,” *Physical review letters*, vol. 103, no. 11, p. 113602, 2009. [Online]. Available: <https://journals.aps.org/prl/abstract/10.1103/PhysRevLett.103.113602>
 - [42] J. Liu and H. Zhou, “Reliability modeling of nisq-era quantum computers,” in *2020 IEEE international symposium on workload characterization (IISWC)*. IEEE, 2020, pp. 94–105.
 - [43] N. Maring, A. Fyrrillas, M. Pont, E. Ivanov, P. Stepanov, N. Margaria, W. Hease, A. Pishchagin, A. Lemaître, I. Sagnes, T. H. Au, S. Boissier, E. Bertasi, A. Baert, M. Valdivia, M. Billard, O. Acar, A. Briussel, R. Mezher, S. C. Wein, A. Salavrakos, P. Sinnott, D. A. Fioretto, P.-E. Emeriau, N. Belabas, S. Mansfield, P. Senellart, J. Senellart, and N. Somaschi, “A versatile single-photon-based quantum computing platform,” *Nature Photonics*, vol. 18, no. 6, pp. 603–609, Jun. 2024, publisher: Nature Publishing Group. [Online]. Available: <https://www.nature.com/articles/s41566-024-01403-4>
 - [44] F. Marsili, V. B. Verma, J. A. Stern, S. Harrington, A. E. Lita, T. Gerrits, I. Vayshenker, B. Baek, M. D. Shaw, R. P. Mirin *et al.*, “Detecting single infrared photons with 93% system efficiency,” *Nature Photonics*, vol. 7, no. 3, pp. 210–214, 2013.
 - [45] S. Maurya and S. Tannu, “Compaqt: Compressed waveform memory architecture for scalable qubit control,” in *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2022, pp. 1059–1077.
 - [46] Z. Mo, Y. Li, A. Pawar, X. Tang, J. Yang, and Y. Zhang, “Fcm: A fusion-aware wire cutting approach for measurement-based quantum computing,” in *Proceedings of the 61st ACM/IEEE Design Automation Conference*, 2024, pp. 1–6.
 - [47] P. Murali, J. M. Baker, A. Javadi-Abhari, F. T. Chong, and M. Martonosi, “Noise-adaptive compiler mappings for noisy intermediate-scale quantum computers,” in *Proceedings of the twenty-fourth international conference on architectural support for programming languages and operating systems*, 2019, pp. 1015–1029.
 - [48] J. L. O’Brien, A. Furusawa, and J. Vučković, “Photonic quantum technologies,” *Nature photonics*, vol. 3, no. 12, pp. 687–695, 2009.
 - [49] A. Olivo and F. Grosshans, “Ancilla-assisted linear optical bell measurements and their optimality,” *Physical Review A*, vol. 98, no. 4, p. 042323, 2018.
 - [50] T. Patel and D. Tiwari, “Veritas: accurately estimating the correct output on noisy intermediate-scale quantum computers,” in *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 2020, pp. 1–16.
 - [51] L. A. Pettersson, A. S. Sørensen, and S. Paesani, “Deterministic Generation of Concatenated Graph Codes from Quantum Emitters,” *PRX Quantum*, vol. 6, no. 1, p. 010305, Jan. 2025. [Online]. Available: <https://link.aps.org/doi/10.1103/PRXQuantum.6.010305>
 - [52] PsiQuantum team, K. Alexander, A. Benyamini, D. Black, D. Bonneau, S. Burgos, B. Burrage, H. Cable, G. Campbell, G. Catalano, A. Ceballos, C.-M. Chang, S. S. Choudhury, C. J. Chung, F. Danesh, T. Dauer, M. Davis, E. Dudley, P. Er-Xuan, J. Fargas, A. Farsi, C. Fenrich, J. Frazer, M. Fukami, Y. Ganesan, G. Gibson, M. Gimeno-Segovia, S. Goeldi, P. Goley, R. Haismaier, S. Halimi, P. Hansen, S. Hardy, J. Horng, M. House, H. Hu, M. Jadidi, V. Jain, H. Johansson, T. Jones, V. Kamineni, N. Kelez, R. Koustuban, G. Kovall, P. Kroger, N. Kumar, Y. Liang, N. LiCausi, D. Llewellyn, K. Lokovic, M. Lovelady, V. R. Manfrinato, A. Melnichuk, G. Mendoza, B. Moores, S. Mukherjee, J. Munns, F.-X. Musalem, F. Najafi, J. L. O’Brien, J. E. Ortmann, S. Pai, B. Park, H.-T. Peng, N. Penthorn, B. Peterson, G. Peterson, M. Poush, G. J. Pryde, T. Ramprasad, G. Ray, A. V. Rodriguez, B. Roxworthy, T. Rudolph, D. J. Saunders, P. Shadbolt, D. Shah, A. Bahgat Shehata, H. Shin, J. Sinsky, J. Smith, B. Sohn, Y.-I. Sohn, G. Son, M. C. M. M. Souza, C. Sparrow, M. Staffaroni, C. Stavrakas, V. Sukumaran, D. Tamborini, M. G. Thompson, K. Tran, M. Triplett, M. Tung, A. Veitia, A. Vert, M. D. Vidrighin, I. Vorobeichik, P. Weigel, M. Wingert, J. Wooding, and X. Zhou, “A manufacturable platform for photonic quantum computing,” *Nature*, vol. 641, no. 8064, pp. 876–883, May 2025. [Online]. Available: <https://www.nature.com/articles/s41586-025-08820-7>
 - [53] R. Raussendorf and H. J. Briegel, “A one-way quantum computer,” *Physical review letters*, vol. 86, no. 22, p. 5188, 2001. [Online]. Available: <https://journals.aps.org/prl/abstract/10.1103/PhysRevLett.86.5188>
 - [54] X. Ren, Y. Huang, Z. Liang, and A. Barbalace, “A scalable and robust compilation framework for emitter-photonic graph state,” *arXiv preprint arXiv:2503.16346*, 2025. [Online]. Available: <https://arxiv.org/abs/2503.16346>
 - [55] —, “A scalable and robust compilation framework for emitter-photonic graph state,” in *2025 62nd ACM/IEEE Design Automation Conference (DAC)*, 2025, pp. 1–7.
 - [56] T. Rudolph, “Why I am optimistic about the silicon-photonics route to quantum computing,” *APL Photonics*, vol. 2, no. 3, p. 030901, Mar. 2017. [Online]. Available: <https://pubs.aip.org/app/article/2/3/030901/122954/Why-I-am-optimistic-about-the-silicon-photonics>
 - [57] A. Russo, E. Barnes, and S. E. Economou, “Photonic graph state generation from quantum dots and color centers for quantum communications,” *Physical Review B*, vol. 98, no. 8, p. 085303, Aug. 2018, arXiv:1801.02754 [cond-mat]. [Online]. Available: <http://arxiv.org/abs/1801.02754>
 - [58] C. Schön, E. Solano, F. Verstraete, J. I. Cirac, and M. M. Wolf, “Sequential generation of entangled multiqubit states,” *Physical review letters*, vol. 95, no. 11, p. 110503, 2005. [Online]. Available: <https://journals.aps.org/prl/abstract/10.1103/PhysRevLett.95.110503>
 - [59] I. Schwartz, D. Cogan, E. R. Schmidgall, Y. Don, L. Gantz, O. Kenneth, N. H. Lindner, and D. Gershoni, “Deterministic generation of a cluster state of entangled photons,” *Science*, vol. 354, no. 6311, pp. 434–437, 2016. [Online]. Available: <https://doi.org/10.1126/science.aah4758>
 - [60] Y. Shi, N. Leung, P. Gokhale, Z. Rossi, D. I. Schuster, H. Hoffmann, and F. T. Chong, “Optimized compilation of aggregated instructions for realistic quantum computers,” in *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, 2019, pp. 1031–1044.
 - [61] S. Stein, S. Xu, A. W. Cross, T. J. Yoder, A. Javadi-Abhari, C. Liu, K. Liu, Z. Zhou, C. Guinn, Y. Ding *et al.*, “Hetec: Architectures for heterogeneous quantum error correction codes,” in *Proceedings of the 30th ACM International Conference on Architectural Support for*

- Programming Languages and Operating Systems, Volume 2*, 2025, pp. 515–528.
- [62] S. S. Tannu and M. Qureshi, “Ensemble of diverse mappings: Improving reliability of quantum computers by orchestrating dissimilar mistakes,” in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, 2019, pp. 253–265.
- [63] S. S. Tannu and M. K. Qureshi, “A case for variability-aware policies for nisq-era quantum computers,” *arXiv preprint arXiv:1805.10224*, 2018.
- [64] —, “Not all qubits are created equal: A case for variability-aware policies for nisq-era quantum computers,” in *Proceedings of the twenty-fourth international conference on architectural support for programming languages and operating systems*, 2019, pp. 987–999.
- [65] J. D. Teoh, P. Winkel, H. K. Babla, B. J. Chapman, J. Claes, S. J. de Graaf, J. W. O. Garmon, W. D. Kalfus, Y. Lu, A. Maiti, K. Sahay, N. Thakur, T. Tsunoda, S. H. Xue, L. Frunzio, S. M. Girvin, S. Puri, and R. J. Schoelkopf, “Dual-rail encoding with superconducting cavities,” *Proceedings of the National Academy of Sciences*, vol. 120, no. 22, p. e2221736120, 2023.
- [66] P. Thomas, L. Ruscio, O. Morin, and G. Rempe, “Fusion of deterministically generated photonic graph states,” *Nature*, vol. 629, no. 8012, pp. 567–572, 2024. [Online]. Available: <https://www.nature.com/articles/s41586-024-07357-5>
- [67] T. Tomesh, P. Gokhale, V. Omole, G. S. Ravi, K. N. Smith, J. Vizslai, X.-C. Wu, N. Hardavellas, M. R. Martonosi, and F. T. Chong, “Supermarq: A scalable quantum benchmark suite,” in *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2022, pp. 587–603.
- [68] M. Varnava, D. E. Browne, and T. Rudolph, “Loss Tolerance in One-Way Quantum Computation via Counterfactual Error Correction,” *Physical Review Letters*, vol. 97, no. 12, p. 120501, Sep. 2006. [Online]. Available: <https://link.aps.org/doi/10.1103/PhysRevLett.97.120501>
- [69] S. Vittal, P. Das, and M. Qureshi, “Astrea: Accurate quantum error-decoding via practical minimum-weight perfect-matching,” in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, 2023, pp. 1–16.
- [70] S. Xu, C. T. Hann, B. Foxman, S. M. Girvin, and Y. Ding, “Systems architecture for quantum random access memory,” in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, 2023, pp. 526–538.
- [71] Y. Zhan, P. Hilaire, E. Barnes, S. E. Economou, and S. Sun, “Performance analysis of quantum repeaters enabled by deterministically generated photonic graph states,” *Quantum*, vol. 7, p. 924, Feb. 2023, publisher: Verein zur Förderung des Open Access Publizierens in den Quantenwissenschaften. [Online]. Available: <https://quantum-journal.org/papers/q-2023-02-16-924/>
- [72] H. Zhang, J. Ruan, H. Shapourian, R. R. Kompella, and Y. Ding, “OnePerc: A Randomness-aware Compiler for Photonic Quantum Computing,” in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, ser. ASPLOS ’24, vol. 3. New York, NY, USA: Association for Computing Machinery, 2024, pp. 738–754. [Online]. Available: <https://dl.acm.org/doi/10.1145/3620666.3651372>
- [73] —, “Oneperc: A randomness-aware compiler for photonic quantum computing,” in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, 2024, pp. 738–754.
- [74] H. Zhang, J. Ruan, D. Tullsen, Y. Ding, A. Li, and T. Humble, “Oneadapt: Resource-adaptive compilation of measurement-based quantum computing for photonic hardware,” in *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO ’25. New York, NY, USA: Association for Computing Machinery, 2025, p. 733–748. [Online]. Available: <https://doi.org/10.1145/3725843.3756100>
- [75] H. Zhang, A. Wu, Y. Wang, G. Li, H. Shapourian, A. Shabani, and Y. Ding, “Oneq: A compilation framework for photonic one-way quantum computation,” in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, ser. ISCA ’23. New York, NY, USA: Association for Computing Machinery, 2023. [Online]. Available: <https://doi.org/10.1145/3579371.3589047>
- [76] —, “Oneq: A compilation framework for photonic one-way quantum computation,” in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, 2023, pp. 1–14.
- [77] J. Zhou, Y. Liu, Y. Shi, A. Javadi-Abhari, and G. Li, “Bosehdral: Compiler optimization for bosonic quantum computing,” in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2024, pp. 261–276.
- [78] S.-C. Zhuang, B. Li, M.-Y. Zheng, Y.-X. Zeng, H.-N. Wu, G.-B. Li, Q. Yao, X.-P. Xie, Y.-H. Li, H. Qin *et al.*, “Ultrabright entanglement based quantum key distribution over a 404 km optical fiber,” *Physical Review Letters*, vol. 134, no. 23, p. 230801, 2025. [Online]. Available: <https://journals.aps.org/prl/abstract/10.1103/PhysRevLett.134.230801>